

One-Click RTL-to-GDSII Automation Framework

for Single-Clock Designs

Script User Guide

Prepared by:

Md. Irfan

Roll No: 25EC4211

Under the Guidance of:

Dr. Hemanta Kumar Mondal

[Assistant Professor, Department of ECE]

Tool Chain:

Cadence Genus (Synthesis) + Cadence Innovus (Physical Design)

PDK: SCL 180nm

[May, 2026]



Department of Electronics and Communication Engineering

राष्ट्रीय प्रौद्योगिकी संस्थान दुर्गापुर

National Institute of Technology Durgapur

**Mahatma Gandhi Avenue, Durgapur-713209, West Bengal
INDIA**

1. Overview

This script takes your RTL Verilog files and produces a complete GDSII layout in a single command. It covers synthesis, floorplan, power planning, placement, CTS, and routing. It runs on Cadence Genus and Innovus using the SCL 180nm PDK.

What the script does, in order:

- Runs Genus to synthesise your RTL into a gate-level netlist
- Generates a padframe (.io file) based on your design ports
- Creates a padframe-wrapped netlist for Innovus
- Runs all Innovus PD stages: floorplan, power plan, placement, CTS, routing
- Streams out the final GDSII and organises outputs into numbered folders

Output folders created:

Folder	Contents
1_synthesis	Gate-level netlist, SDC, synthesis reports
2_floorplan	Padframe files, floorplan checkpoint (.enc)
3_powerplan	Power ring/stripe checkpoint, power report
4_placement	Placed netlist and timing report
5_cts	Post-CTS netlist, clock and timing reports
6_routing	Final routed netlist, area/power/timing reports
7_GDSII	Final GDSII layout file

2. Before You Start

2.1 Requirements

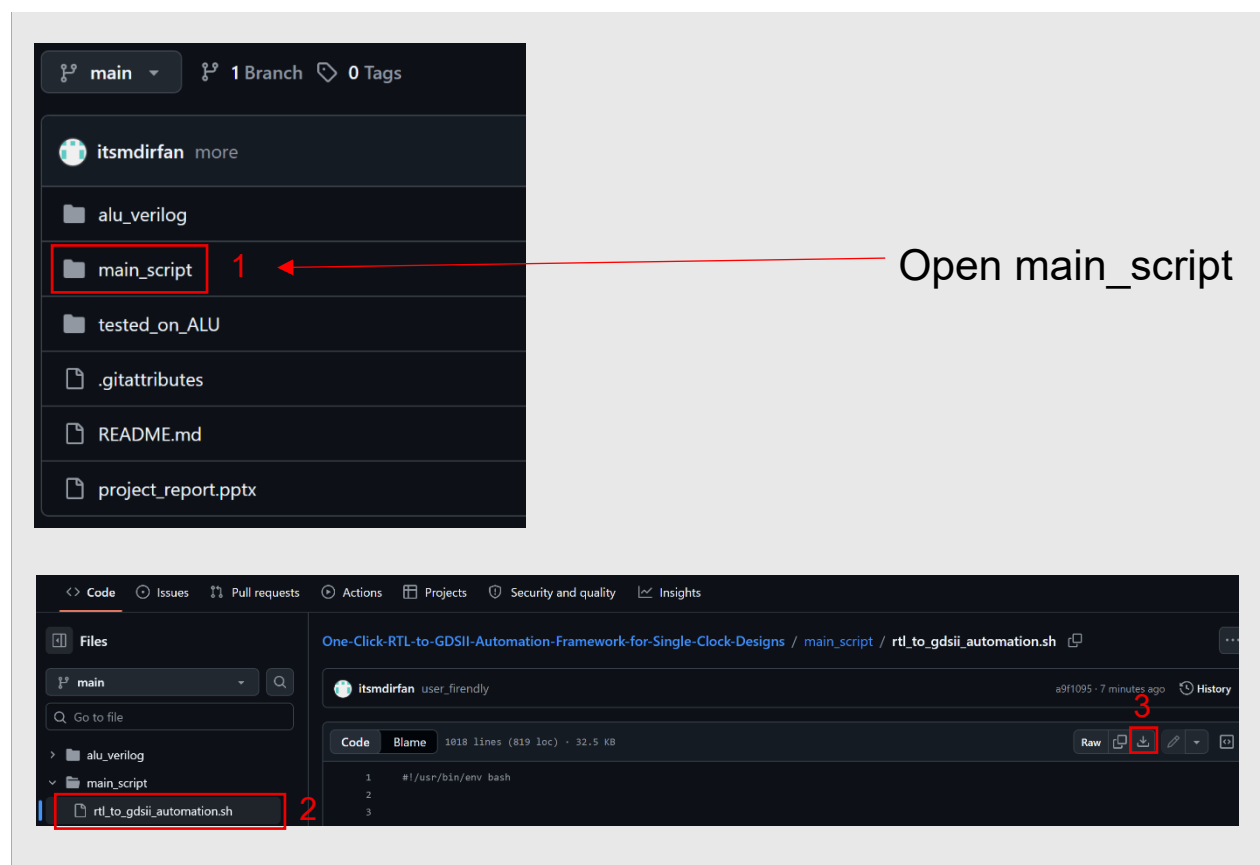
- Cadence Genus and Innovus installed and licensed on the lab PC
- SCL 180nm PDK at: /home/vlsiXX/Downloads/scl_pdk
- Cadence cshrc at: /home/vlsiXX/c2s/cadence/install/cshrc
- You are running from your home directory (/home/vlsiXX/...)
- Single-clock design only

2.2 Download the Automation Script

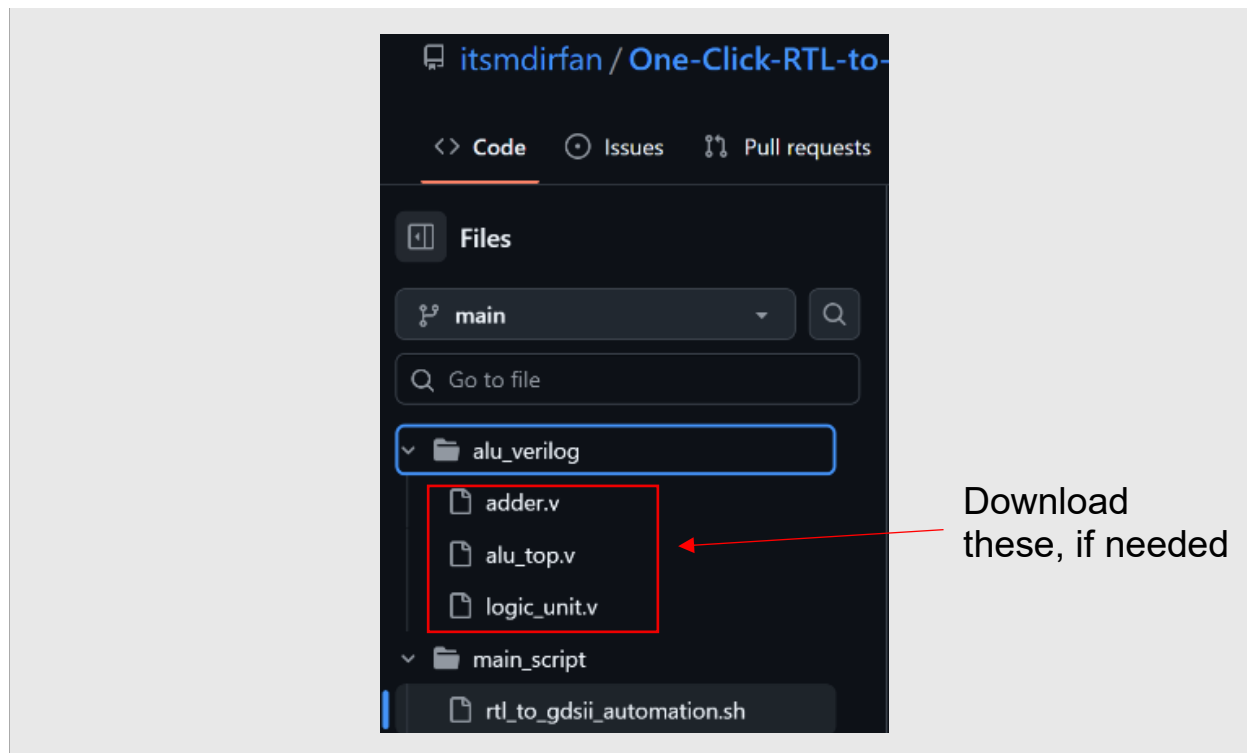
The script is available on GitHub. Go to the link below and download rtl_to_gdsii_automation.sh.

GitHub link:

[<https://github.com/itsmdirfan/One-Click-RTL-to-GDSII-Automation-Framework-for-Single-Clock-Designs>]



If you want to test on the same design used in this guide (8-bit ALU), the three Verilog files are also available at the same repository. Similarly download `alu_top.v`, `adder.v`, and `logic_unit.v`.



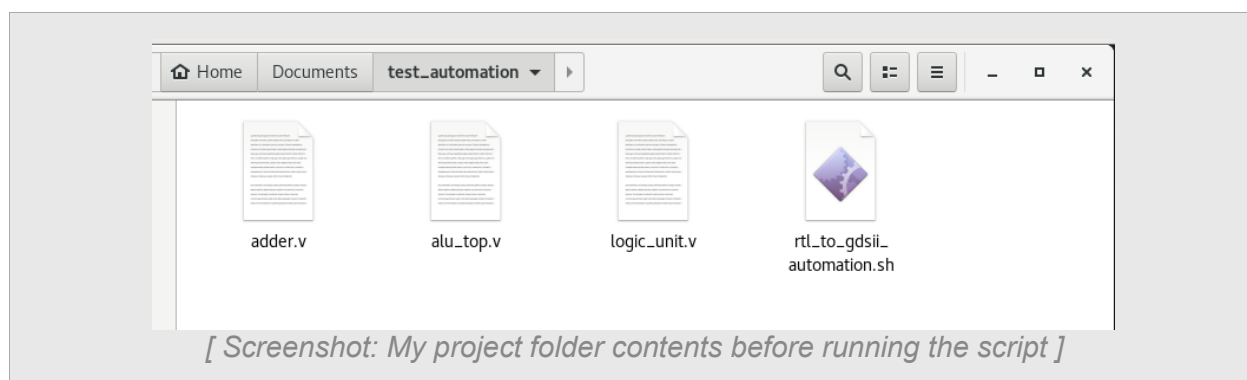
2.3 Prepare Your Project Folder

Create a new empty folder for your project. Put the following into that folder:

- All your Verilog (.v) source files
- The automation script: `rtl_to_gdsii_automation.sh`

For the ALU test run, your folder should look like this:

```
my_project/
  alu_top.v
  adder.v
  logic_unit.v
  rtl_to_gdsii_automation.sh
```



3. Configuring the Script

Open the script in any text editor. At the very top you will find the USER CONFIGURATION section. This is the only part you need to edit. All steps below show how to fill it in using the ALU design as an example.

```
# =====
#                               USER CONFIGURATION
# =====

# RTL files to synthesize (space-separated); all must be in this directory
# single file   : RTL_FILES="alu.v"
# multiple files: RTL_FILES="alu.v datapath.v controller.v"
RTL_FILES="source.v"

# Top module name (as declared in RTL)
TOP_MODULE="source"

# Clock port name (as in RTL), period in ps, I/O delays in ns
CLK_NAME="clk"
CLK_PERIOD_PS=5000      # 5000 ps = 5 ns = 200 MHz
INPUT_DELAY_NS=1
OUTPUT_DELAY_NS=1

# Synthesis effort: low / medium / high
SYN_EFFORT="high"

# If placement fails due to core-limited design, increase dummy IO count here.
# Must be >= actual number of IO ports; will be rounded up to next multiple of 4.
# Leave 0 to use the auto-calculated count (default).
USER_IO_COUNT=0

# Fallback vlsi PC number if auto-detection fails
DEFAULT_VLSI_PC=12

# expects: /home/vlsi<N>/Downloads/scl_pdk - override if PDK is elsewhere
PDK_BASE_OVERRIDE=""

# expects: /home/vlsi<N>/c2s/cadence/install/cshrc - override if Cadence is not installed in c2s
CADENCE_CSH_OVERRIDE=""
```

[Screenshot: USER CONFIGURATION section]

3.1 RTL Files and Top Module

Set RTL_FILES to a space-separated list of all your Verilog files. Set TOP_MODULE to your top-level module name exactly as written in the Verilog.

For the ALU design:

```
RTL_FILES="alu_top.v adder.v logic_unit.v"
TOP_MODULE="alu_top"
```

Single file example:

```
RTL_FILES="mydesign.v"
TOP_MODULE="mydesign"
```

Note

TOP_MODULE must match the module name in the Verilog exactly (case-sensitive).

```
# RTL files to synthesize (space-separated); all must be in this directory
# single file : RTL_FILES="alu.v"
# multiple files: RTL_FILES="alu.v datapath.v controller.v"
RTL_FILES="alu_top.v adder.v logic_unit.v"

# Top module name (as declared in RTL)
TOP_MODULE="alu_top"
```

[Screenshot: RTL_FILES and TOP_MODULE filled in for ALU design]

3.2 Clock Constraints

Set the clock port name, period, and I/O delays to match your design.

Variable	What it controls	ALU value
CLK_NAME	Clock port name in RTL	CLK_NAME="clk"
CLK_PERIOD_PS	Clock period in picoseconds	CLK_PERIOD_PS=5000 (5 ns)
INPUT_DELAY_NS	Input arrival delay (ns)	INPUT_DELAY_NS=1
OUTPUT_DELAY_NS	Output required delay (ns)	OUTPUT_DELAY_NS=1

For the ALU design, leave all clock values at their defaults. To target a different frequency on your own design, change CLK_PERIOD_PS.

```
module alu_top (
    input      clk,
    input [7:0] a,
    input [7:0] b,
    input      cin,
    input [2:0] opcode,
    output reg [7:0] result,
    output reg      cout,
    output      zero
);
```

[Screenshot: Clock port name in my design (clk)]

```
# Clock port name (as in RTL), period in ps, I/O delays in ns
CLK_NAME="clk"
CLK_PERIOD_PS=5000      # 5000 ps = 5 ns = 200 MHz
INPUT_DELAY_NS=1
OUTPUT_DELAY_NS=1
```

[Screenshot: Clock constraint lines filled in for ALU design]

3.3 Synthesis Effort

SYN_EFFORT controls quality vs. runtime in Genus.

- low: fastest, lowest quality
- medium: balanced
- high: best result, slowest (use this for final runs)

For the ALU design:

```
SYN_EFFORT="high"
```

3.4 VLSI PC Number

You only need to change DEFAULT_VLSI_PC if the script prints a warning that it could not detect the PC number. For the ALU test run on your lab PC, leave it as is.

```
DEFAULT_VLSI_PC=12 # only change if auto-detect fails
```

3.5 PDK and Cadence Paths

By default the paths are built from your PC number:

- PDK: /home/vlsiXX/Downloads/scl_pdk
- Cadence: /home/vlsiXX/c2s/cadence/install/cshrc

For the ALU test run, leave both override variables blank. Only fill them in if your installation is at a different location.

```
PDK_BASE_OVERRIDE=""  
CADENCE_CSH_OVERRIDE=""
```

3.6 Core-Limited Design: USER_IO_COUNT (optional)

Skip this for the ALU test run. Set USER_IO_COUNT=0 and move on.

What this is for: the script adds 4 power pads (VDD, VDDO, VSS, VSSO) to your design IOs automatically. The total (your IOs + 4 power pads) is rounded up to the next multiple of 4 so each side of the chip gets an equal number of pads. Any remaining slots are filled with dummy pads.

Example with 11 IOs:

- 11 signal IOs + 4 power pads = 15 total
- 15 rounded up to 16 (next multiple of 4)
- 1 dummy pad added, 4 pads per side

The problem: if your design has a large cell area but few IOs, the chip ends up too small to place all the standard cells. Placement will fail.

The fix: set USER_IO_COUNT to a number larger than your actual IO count. The script adds more dummy pads, making the chip larger.

```
USER_IO_COUNT=0 # 0 = automatic
```

Actual IOs	USER_IO_COUNT	Result
11	0 (auto)	15 + 1 dummy = 16 total, 4 per side
11	22	22 + 4 power = 26, rounded to 28, 13 dummies, 7 per side
11	36	36 + 4 power = 40, 25 dummies, 10 per side

Note

USER_IO_COUNT must be $\geq$ your actual IO count. If you enter a smaller number the script ignores it.

The extra slots are filled with dummy pads (pc3d01) that carry no signal.

For the ALU, 22 signal IOs + 4 power = 26, rounded to 28, 7 per side. No override needed.

```
# =====
#                               USER CONFIGURATION
# =====

# RTL files to synthesize (space-separated); all must be in this directory
# single file   : RTL_FILES="alu.v"
# multiple files: RTL_FILES="alu.v datapath.v controller.v"
RTL_FILES="alu_top.v adder.v logic_unit.v"

# Top module name (as declared in RTL)
TOP_MODULE="alu_top"

# Clock port name (as in RTL), period in ps, I/O delays in ns
CLK_NAME="clk"
CLK_PERIOD_PS=5000      # 5000 ps = 5 ns = 200 MHz
INPUT_DELAY_NS=1
OUTPUT_DELAY_NS=1

# Synthesis effort: low / medium / high
SYN_EFFORT="high"

# If placement fails due to core-limited design, increase dummy IO count here.
# Must be  $\geq$  actual number of IO ports; will be rounded up to next multiple of 4.
# Leave 0 to use the auto-calculated count (default).
USER_IO_COUNT=0

# Fallback vlsi PC number if auto-detection fails
DEFAULT_VLSI_PC=12

# expects: /home/vlsi<N>/Downloads/sc1_pdk - override if PDK is elsewhere
PDK_BASE_OVERRIDE=""

# expects: /home/vlsi<N>/c2s/cadence/install/cshrc - override if Cadence is not installed in c2s
CADENCE_CSH_OVERRIDE=""
```

[Screenshot: Complete USER CONFIGURATION section filled in for ALU]

4. Running the Script

4.1 Open a Terminal in Your Project Folder

Right-click on your project folder in the file manager and select Open Terminal Here. This opens a terminal already pointing to that folder.

4.2 Give Execution Permission

Before running for the first time, give the script execute permission:

```
chmod +x rtl_to_gdsii_automation.sh
```

```
[vlsi3@ece test_automation]$ chmod +x rtl_to_gdsii_automation.sh
```

[Screenshot: Terminal: chmod command executed]

4.3 Run the Script

```
bash rtl_to_gdsii_automation.sh
```

```
[vlsi3@ece test_automation]$ bash rtl_to_gdsii_automation.sh
```

[Screenshot: Running script]

Note

Do not close the terminal while the script is running. Genus and Innovus open and close automatically. The full run takes time depending on design size.

```
VLSI PC (auto-detected): 3
=== FULL RTL to GDSII AUTOMATION START ===
Project Root: /home/vlsi3/Documents/test_automation
Cadence cshrc: /home/vlsi3/c2s/cadence/install/cshrc
Using vlsi PC number: 3
```

```
--- Starting Genus Synthesis ---
Executing Genus... This command will wait until synthesis is complete.
```

```
Generated modified netlist: alu_top_netlist_modified.v
Inputs parsed: 21, Outputs parsed: 10
Pad instances added: 31
```

```
#####Streamout is finished!

*** Memory Usage v#1 (Current mem = 1983.637M, initial mem = 483.879M) ***
*** Message Summary: 1692 warning(s), 0 error(s)

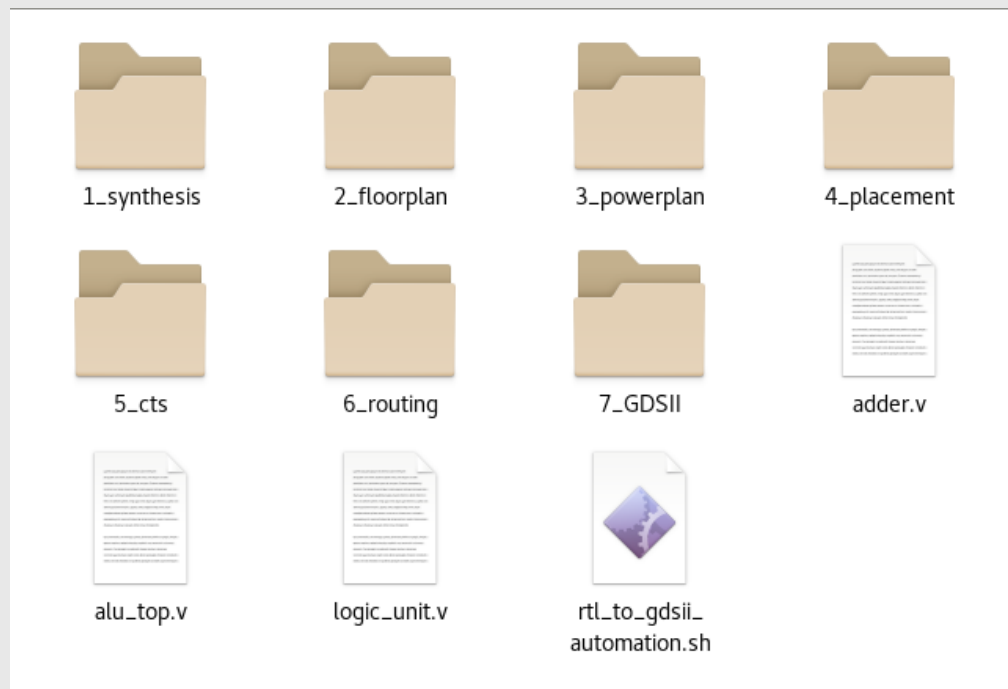
--- Ending "Innovus" (totcpu=0:00:35.9, real=0:00:45.0, mem=1983.6M) ---
```

[Screenshot: Some terminal outputs]

4.4 Successful Completion

When the script finishes, your project folder will have these sub-folders:

```
1_synthesis/    2_floorplan/    3_powerplan/
4_placement/    5_cts/          6_routing/      7_GDSII/
```



[Screenshot: Project folder with all 7 output sub-folders created]

5. Checking the Outputs

5.1 Reports

Each stage produces text reports. Key ones to check:

File	Location	What to check
synth_report_timing.rpt	1_synthesis/	Setup/hold slack
synth_report_area.rpt	1_synthesis/	Cell count and total area
synth_report_power.rpt	1_synthesis/	Total power estimate
timing_routing.rpt	6_routing/	Final timing after routing
area_routing.rpt	6_routing/	Final routed area
power_routing.rpt	6_routing/	Final power after routing

```

g1723__1705/ZN      aoim21d1      3 11.1 369 +298 3124 R
g1719__3680/B1      aoim21d1      3 11.1 369 +298 3124
g1719__3680/ZN      aoim21d1      3 11.1 369 +298 3422 R
g1716__8428/B1      aoim21d1      3 11.1 369 +298 3422
g1716__8428/Z       aoim22d1      1 5.1 250 +265 3688 R
g1712__6260/C1      aoim22d1      1 5.1 250 +265 3688
g1712__6260/ZN      oai211d1      1 1.9 192 +122 3809 F
result_reg[7]/D <<< dfnrq1      +0 3809
result_reg[7]/CP     setup          100 +221 4031 R
-----
(clock clk)          capture          5000 R
-----
Timing slack :      969ps
Start-point : b[0]
End-point   : result_reg[7]/D

```

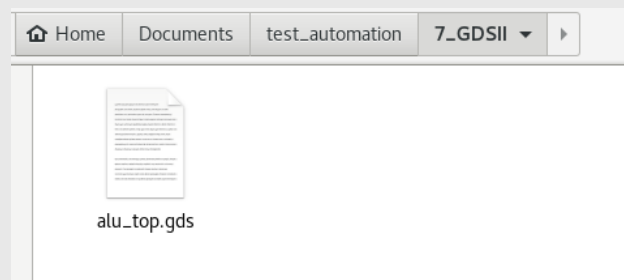
[Screenshot: synth_report_timing.rpt open in text editor]

5.2 GDSII File

The final layout is at:

```
7_GDSII/<TOP_MODULE>.gds
```

This can be opened in Virtuoso to inspect the layout.



[Screenshot: 7_GDSII folder showing .gds file]

5.3 Loading a Checkpoint in Innovus

Each PD stage saves a .enc checkpoint. You can reload any stage in Innovus to inspect or continue from that point.

To load the routing result as an example:

1. Open a terminal and go to the 6_routing folder
2. Launch Innovus:

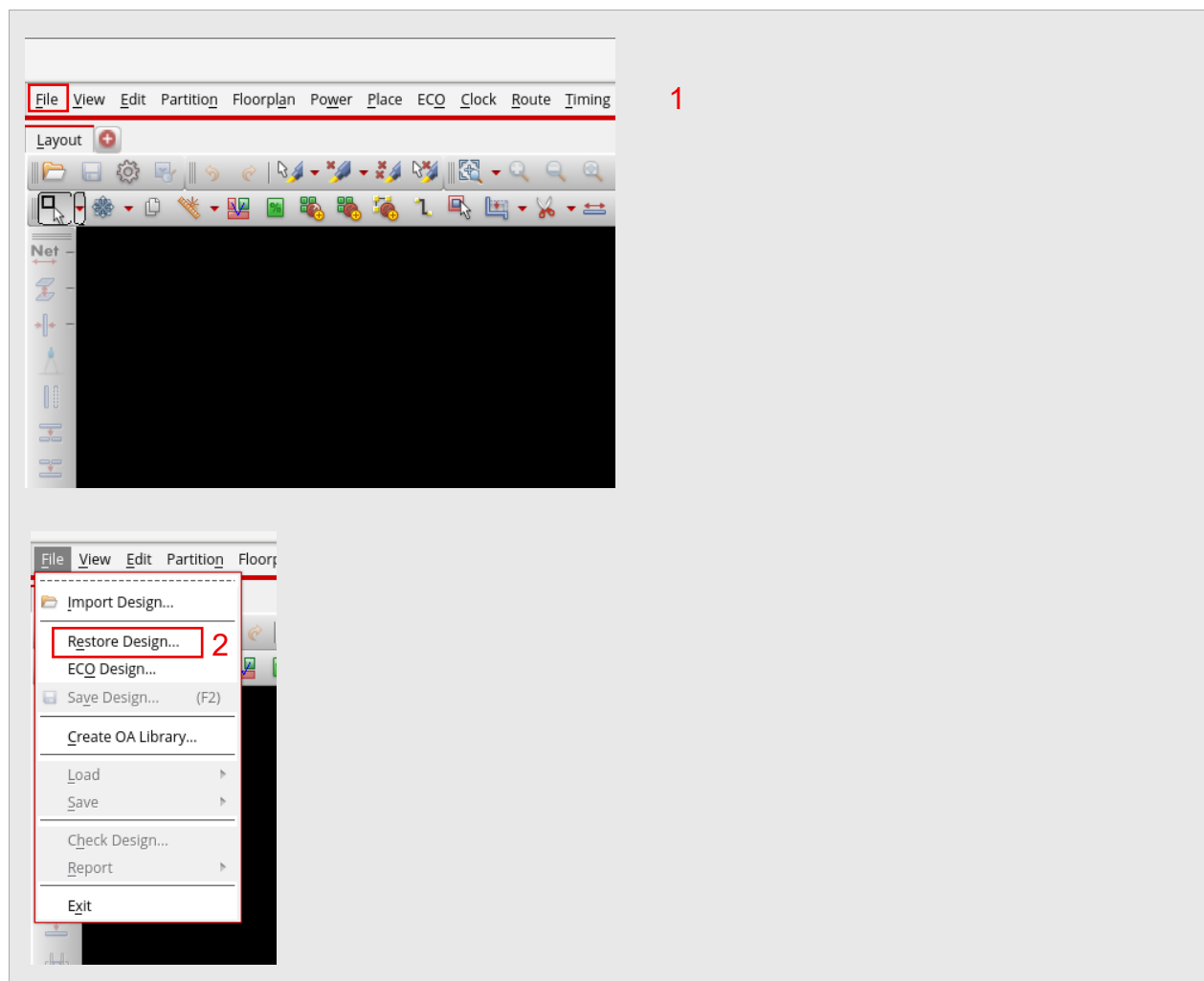
```
[vlsi3@ece 6_routing]$ csh
[vlsi3@ece 6_routing]$ source /home/vlsi3/c2s/cadence/install/cshrc

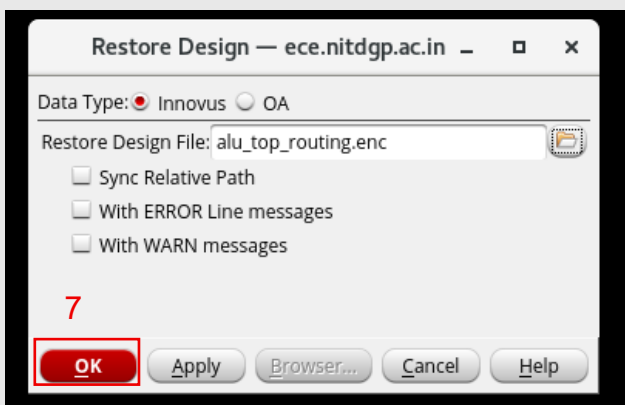
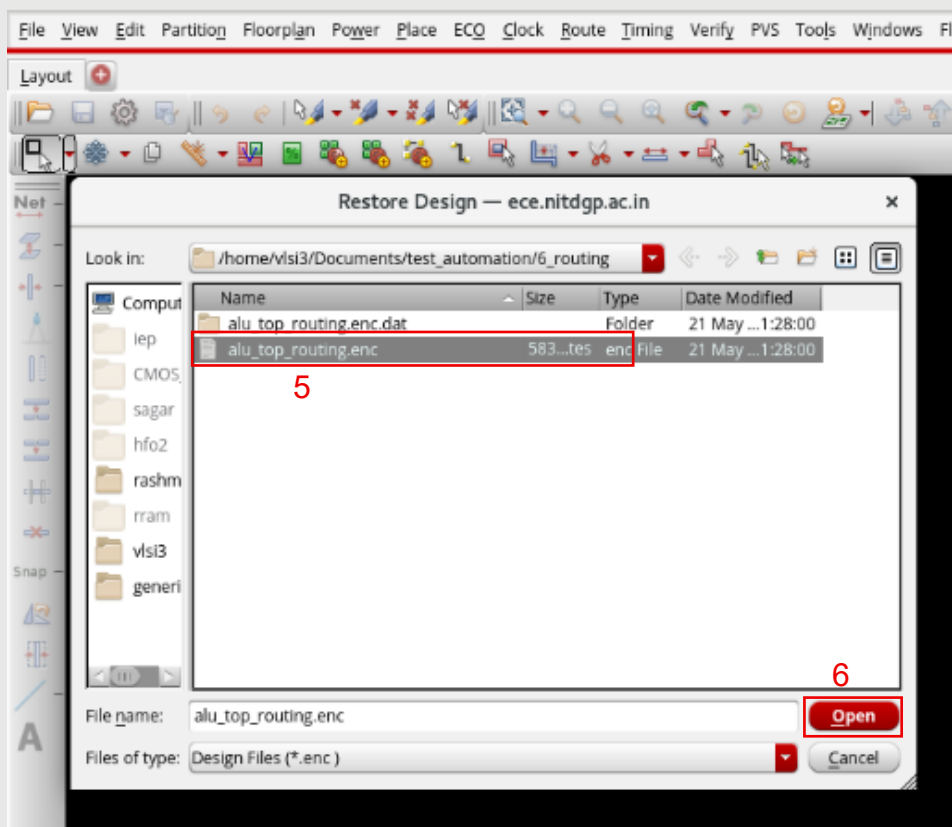
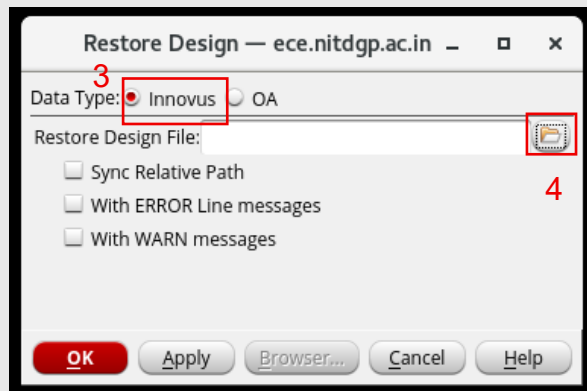
Welcome to Cadence tools Suite

[vlsi3@ece 6_routing]$ innovus
```

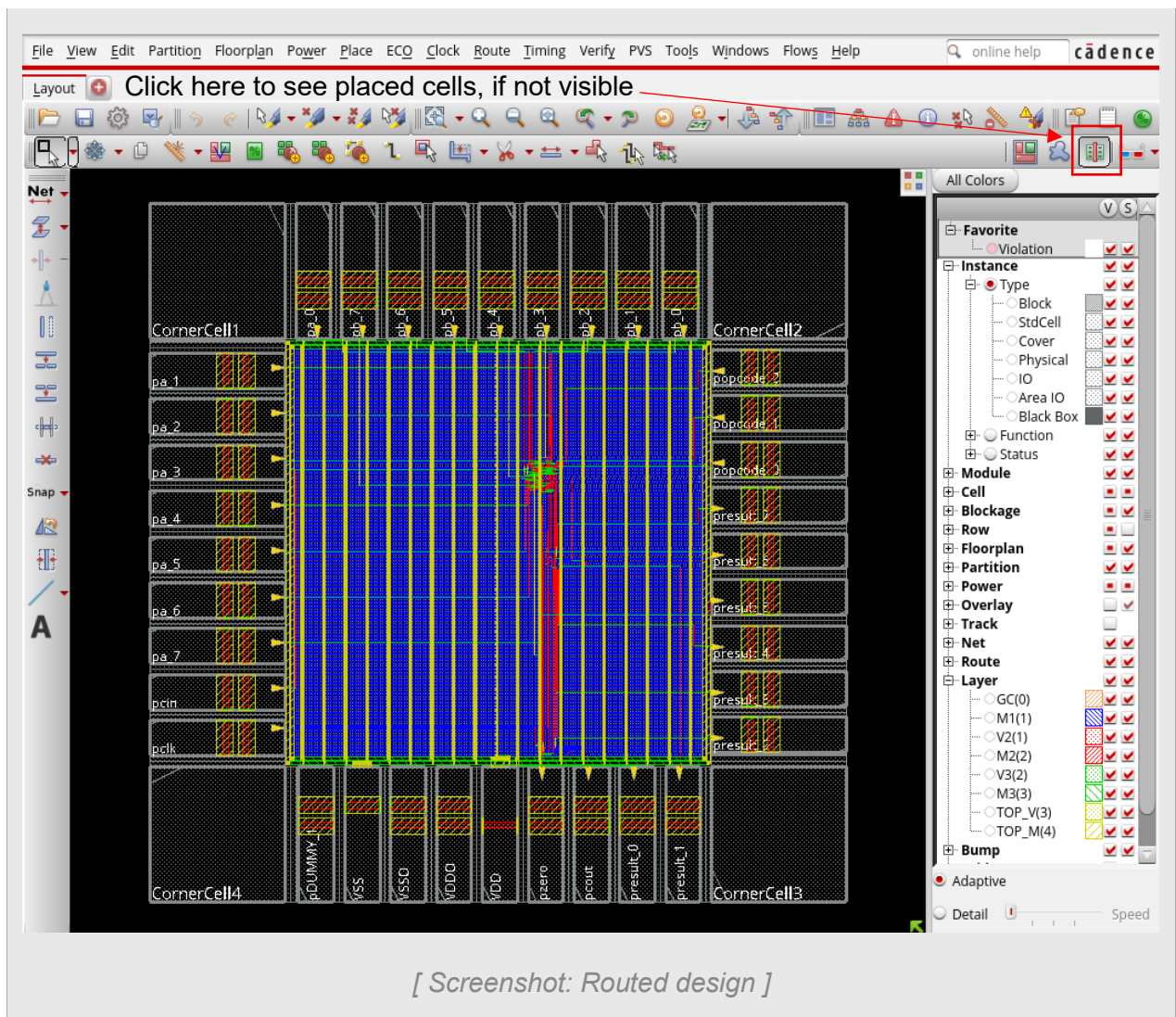
[Screenshot: Manually source Cadence and invoke Innovus]

3. Load .enc into Innovus





[Screenshot: Steps to load .enc into Innovus]



6. Troubleshooting

6.1 Library not found at Genus start

- Check the PDK is at /home/vlsiXX/Downloads/scl_pdk
- If not, set PDK_BASE_OVERRIDE to the correct path
- Confirm the VLSI PC number detected at script start is correct

6.2 Genus: unresolved module

- A file is missing from RTL_FILES
- A module name in an instantiation does not match its declaration
- Check all filenames are spelled correctly and all files are in the project folder

6.3 Timing violation after synthesis

- The clock period may be too tight for this PDK
- Try increasing CLK_PERIOD_PS
- Make sure SYN_EFFORT is set to high

6.4 Placement fails: core area too small

- Your design is core-limited
- Increase USER_IO_COUNT (see Section 3.6)
- Start with double your actual IO count and go higher if needed

6.5 Cadence tools not found

- Check the cshrc path is correct
- If not at the default c2s location, set CADENCE_CSH_OVERRIDE

Contact

If you run into any issue using this script, feel free to reach out on LinkedIn.

LinkedIn: [\[https://www.linkedin.com/in/irfan3\]](https://www.linkedin.com/in/irfan3)

Md. Irfan

Roll No: 25EC4211

Under the Guidance of:

Dr. Hemanta Kumar Mondal

[Assistant Professor, Department of ECE]